

***OczyEdge : Lightweight Vision-Language Embedding
Retrieval for Zero-Shot Checkout System***

Jiaen Suen

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	4
1.3 Definitions, Acronyms, and Abbreviations.....	5
1.4 References.....	6
2. Overall Description.....	7
2.1 Product Perspective.....	7
2.2 Product Functions.....	8
2.3 User Characteristics.....	8
2.4 Constraints.....	8
2.5 Assumptions and Dependencies.....	9
3. Specific Requirements.....	10
3.1 Functional Requirements.....	10
3.2 Non-Functional Requirements.....	11
4. System Design Overview.....	12
4.1 High-Level Architecture (UMLs).....	12
4.2 ML System Design & MLOps.....	15
4.3 Technology Stack.....	17
4.4 GUI Design.....	18
5. Development Estimation and Release Plan.....	19
5.1 Version 1.0 — Minimum Viable Product.....	19
5.2 Version 2.0 — Professional Release.....	20
5.3 Version 3.0 — Mature Release.....	20
6. Testing.....	21
6.1 Testing Objectives and Strategy.....	21
6.2 Test Environment.....	21
6.3 Functional Testing.....	22
6.4 Functional Testing in Show Case.....	22
6.5 Model and Retrieval Evaluation.....	23
6.6 Integration Testing.....	24
6.7 Robustness and Error-Handling Testing.....	25
6.8 Regression and Automated Testing.....	25
7. Version History.....	26

1. Introduction

1.1 Purpose

In retail and foodservice environments, the checkout process has long relied on manual operations. This not only increases labor costs but also easily leads to queue congestion and efficiency bottlenecks during peak hours. Therefore, self-checkout (Self-checkout) and assisted checkout systems have gradually become important solutions for improving operational efficiency. Through human-machine collaboration, these systems accelerate transaction processes and reduce human error rates.

Although mature self-checkout equipment has already existed on the market in recent years—such as the integrated image recognition and sensing systems adopted by large chain supermarkets, which have reached commercially viable levels in terms of accuracy and stability—these systems generally suffer from high implementation costs, strong hardware dependency, and complex deployment. For small and medium-sized businesses (such as bakeries, buffet restaurants, and grocery stores), these solutions are difficult to adopt due to both economic and technical barriers, resulting in limited widespread adoption of automated checkout technologies.

On the other hand, product recognition systems based on traditional computer vision methods (for example, using YOLO series models ([6], [7]) for object detection and classification) have been extensively researched and applied, including previous implementations in buffet checkout systems. However, these approaches typically rely on supervised learning, requiring large amounts of annotated data for model training. Moreover, when new products are added or existing ones change, data must be recollected and models retrained. For merchants lacking engineering resources and data collection capabilities, the high maintenance costs and low scalability of such methods make them difficult to sustain as practical, long-term solutions.

With the advancement of deep learning technologies, particularly the CLIP (Contrastive Language-Image Pretraining) model proposed in 2021 [1], a new direction has been opened for Vision-Language models in cross-modal understanding. These models map images and semantics into a shared embedding space, enabling Zero-shot or One-shot recognition capabilities without the need for retraining on each category. In recent years, related technologies (such as SigLIP [2] and MobileCLIP [3]) have further evolved toward lightweight and practical applications, demonstrating strong potential for real-world deployment.

Based on the above background, this project aims to design and implement a lightweight Vision-Language product recognition system that utilizes embedding retrieval to achieve Zero-shot Product Recognition. The system allows users to build a functional product recognition database by simply uploading minimal product information (such as a single image, name, and price), without requiring model training or complex configuration. With low cost and high usability as core design goals, the system supports deployment on Edge Devices (e.g., Jetson Orin Nano) and provides a clean, intuitive user interface, enabling even non-engineering users to easily set up and maintain the product recognition system. Through this design, we hope to lower the entry barrier for intelligent checkout systems, promote the practical application of self-checkout technology in small and medium-sized commercial settings, and deliver a scalable and practical smart retail solution.

1.2 Scope

In Scope

- **Zero-shot Product Identification Mechanism** : This system utilizes Vision-Language model embeddings to achieve Zero-shot product recognition. During inference, it converts the input image into an embedding vector and compares it with existing product embeddings using cosine similarity to find the best match. This approach significantly reduces maintenance costs for product updates while improving scalability and flexibility in retail environments with frequently changing inventories.
- **Product Data Registration and Management** : The system provides a user-friendly product registration interface, allowing merchants to upload images and input basic information such as name and price. The Vision-Language model automatically generates embeddings and stores them with the product data. This eliminates complex training processes, enabling non-technical users to easily build and maintain the database. Basic update and deletion functions are also supported.
- **Embedding Retrieval and Similarity search** : The system employs Exact Nearest Neighbor Search with cosine similarity to identify the Top-k matching products. This method is efficient for databases of hundreds of items, offering a good balance between accuracy, low maintenance overhead, and fast inference on Edge Devices.
- **Region Proposal** : To improve recognition accuracy, the system first uses a lightweight object detection model to detect and crop the product region from the input image. Only the cropped area is fed into the Vision-Language model for embedding generation. This reduces complexity, speeds up inference, and minimizes background interference, making it well-suited for resource-limited Edge Devices.
- **Self-checkout Process Integration** : The system integrates product recognition into a simplified self-checkout process. After identification, it displays the product name and price for user confirmation. This human-machine collaborative approach improves checkout efficiency and reduces manual errors, while focusing solely on recognition and information display without payment processing.

Out of Scope

The design of this system focuses on validating the feasibility of lightweight Vision-Language embedding retrieval for Zero-shot product recognition. Therefore, its scope is deliberately limited to core research objectives and implementation. This project does not include full commercial POS features such as invoicing, inventory management, membership systems, or business analytics. It also excludes any payment processing integration.

In computer vision, the system does not support high-precision multi-object tracking, complex behavior analysis, or challenging scenarios like severe occlusion and heavy object overlap. It focuses on single-product or simple multi-product recognition. The system is built on pre-trained Vision-Language models with lightweight inference optimizations for Edge Devices, without any large-scale model training. It is designed for single-store deployment and does not address cross-store sharing or large-scale commercial architecture.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
CLIP	Contrastive Language-Image Pretraining.
Edge Device	A computing device with limited resources.
Embedding	A dense vector representation of an image or text that captures semantic meaning in a high-dimensional space.
FAISS	Facebook AI Similarity Search - An open-source library for efficient similarity search and clustering of dense vectors.
Few-shot Learning	A machine learning paradigm where the model can generalize from only a small number of training examples per class.
Inference	The process of using a trained model to make predictions on new, unseen data.
Lightweight Model	A neural network model optimized for low computational cost, memory usage, and latency, suitable for deployment on resource-constrained edge devices.
Product Registration	The process in which a store owner uploads product images, names, and prices to build the product database.
Re-ranking Model	A custom-trained lightweight model used to refine the initial ranking results from vision-language embeddings.
Region Proposal	The process of generating candidate bounding boxes that may contain products in a captured image.
UI	User Interface
Vector Database	A database specialized in storing and querying high-dimensional vector embeddings efficiently.
Vision-Language Model (VLM)	A multimodal model that jointly understands images and natural language, enabling zero-shot recognition.
Zero-shot Learning	The ability of a model to recognize or classify objects it has never seen during training, using only semantic information.

1.4 References

- [1] Radford, Alec, et al. “Learning Transferable Visual Models from Natural Language Supervision.” *ArXiv:2103.00020 [Cs]*, 26 Feb. 2021, arxiv.org/abs/2103.00020.
 - [2] Zhai, Xiaohua, et al. “Sigmoid Loss for Language Image Pre-Training.” *ArXiv.org*, 2023, arxiv.org/abs/2303.15343.
 - [3] Vasu, et al. “MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training.” *ArXiv.org*, 2023, arxiv.org/abs/2311.17049.
 - [4] Hugging Face (CLIP) : https://huggingface.co/docs/transformers/model_doc/clip
 - [5] YOLO-Based Self-Checkout : <https://arxiv.org/html/2407.21308v2>
 - [6] Tian, Yunjie, et al. “YOLOv12: Attention-Centric Real-Time Object Detectors.” *ArXiv.org*, 2025, arxiv.org/abs/2502.12524.
 - [7] “YOLO26: An Analysis of NMS-Free End to End Framework for Real-Time Object Detection.” *Arxiv.org*, 2018, arxiv.org/html/2601.12882v2. Accessed 15 Apr. 2026.
 - [8] Hugging Face : <https://huggingface.co/>
 - [9] Ultralytics : <https://www.ultralytics.com/>
 - [10] PyTorch : <https://pytorch.org/>
 - [11] NanoDet-Plus : <https://github.com/RangiLyu/nanodet>
 - [12] Douze, Matthijs, et al. “The Faiss Library.” *ArXiv.org*, 16 Jan. 2024, arxiv.org/abs/2401.08281
 - [13] Cheng, T., Song, L., Ge, Y., Liu, W., Wang, X., & Shan, Y. (2024). YOLO-World: Real- Time Open-Vocabulary Object Detection. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 16901–16911. <https://arxiv.org/abs/2401.17270>
-

2. Overall Description

2.1 Product Perspective

The system is designed as a modular, edge-deployable product identification and checkout solution that can operate in two main configurations in existing retail environments.



Fig.1. The left image shows a stand-alone edge payment system, and the right image shows an embedded software module suitable for existing POS systems.

(1) Standalone Edge Payment System

In a standalone configuration, the system operates as an independent checkout unit deployed on an edge device, such as a Raspberry Pi, with an integrated camera and compact display. The device performs on-device inference using a lightweight vision-language embedding model, while product identification is achieved through vector similarity search. The system directly handles product detection, price retrieval, and the checkout interface. This setup is suitable for small businesses that lack existing point-of-sale infrastructure, such as bakeries and stationery stores.

(2) Embedded Software Module for Existing POS Systems

In the second configuration, the system is designed as a software module that can be integrated into existing point-of-sale (POS) systems. The identification pipeline functions as a backend AI service and provides APIs for product recognition, embedding retrieval, and similarity matching. It can be integrated with cashier systems, self-checkout kiosks, and inventory management systems. This approach enables retailers to enhance existing workflows without replacing their entire POS infrastructure.

2.2 Product Functions

- Allow users to register new products by uploading product images, names, and prices.
- Allow the system to capture product images using a camera device.
- Allow the system to display the recognized product name and price to the user.
- Allow users to add recognized products to a checkout list.
- Allow the system to calculate the total price of selected products.
- Allow the system to operate on edge devices with limited computational resources.
- Allows for the correction of identification errors or modification of checkout lists during checkout system runtime.

2.3 User Characteristics

The target users of the system are small business owners, such as bakery operators, stationery store owners, and other retailers who require a simple and efficient product identification and checkout solution. Most users are expected to have little or no technical background, with limited experience in machine learning or software development. However, they are generally familiar with basic computer operations, such as using web browsers, uploading files, and interacting with graphical user interfaces.

The system is designed with a strong emphasis on minimizing the learning curve. Users can register products simply by uploading product images along with basic metadata, such as product names and prices. No model training, parameter tuning, or technical configuration is required. During the prototype stage, the system can be distributed through platforms such as GitHub and requires only minimal setup, such as running scripts or installing dependencies. However, the architecture has been designed with future scalability in mind, allowing the system to be packaged as a standalone application that users can easily download and run with minimal installation effort.

In summary, the system is intended to minimize operational complexity and technical barriers, enabling users to quickly adopt and operate the solution without requiring expertise in artificial intelligence or programming.

2.4 Constraints

Technical Constraints

- *The system must be able to run on resource-constrained edge devices with limited CPUs, memory, and no dedicated GPUs.*
- *The system must support real-time or near real-time inference to ensure that latency in the checkout process is within acceptable limits.*
- *The system must run under lightweight model constraints, requiring an efficient model architecture and optimized inference (e.g., ONNX or similar formats).*
- *The system must support offline or low-connectivity environments, and all core functions must be able to operate without a continuous network connection.*

- *The system must be able to handle limited training data, as each product may only have one or a few reference images.*
- *Even as the number of registered products increases, the system must ensure scalable vector retrieval performance.*
- *Prototype implementations may rely on specific frameworks and libraries, which may introduce compatibility limitations between different platforms.*

Deployment Constraints

- *The system must be able to be deployed in small retail environments with very low infrastructure requirements.*
- *The system should minimize installation and configuration work, especially for non-technical users.*
- *Future versions of the system should support packaged distribution to reduce reliance on development environments such as GitHub setups.*

2.5 Assumptions and Dependencies

Assumptions

- *Suppose the user has a camera device capable of capturing product images of sufficient quality for identification.*
- *Assume that users can provide at least one reference image for each product, as well as basic metadata such as product name and price.*
- *Assuming the captured product image is quite clear and without severe obstruction, the system is able to extract meaningful visual features.*
- *The system assumes that the number of products is kept within a manageable range, thus ensuring efficient retrieval.*
- *Assume that the user has basic computer skills, such as operating web page interfaces and uploading files.*
- *For prototype use, it is assumed that the user can perform basic setup steps (e.g., run the program or install the required software packages).*

Dependencies

- *The system relies on a pre-trained visual language model to generate image embeddings.*
- *This system relies on a vector similarity search library for efficient nearest neighbor retrieval.*
- *The system relies on a database system to store product metadata.*
- *The system relies on a backend framework to handle API requests and system logic.*
- *The system relies on the hardware capabilities of edge devices, which directly affects the system's latency and performance.*
- *The system may rely on model transformation and inference frameworks for optimized deployment on edge devices.*

3. Specific Requirements

3.1 Functional Requirements

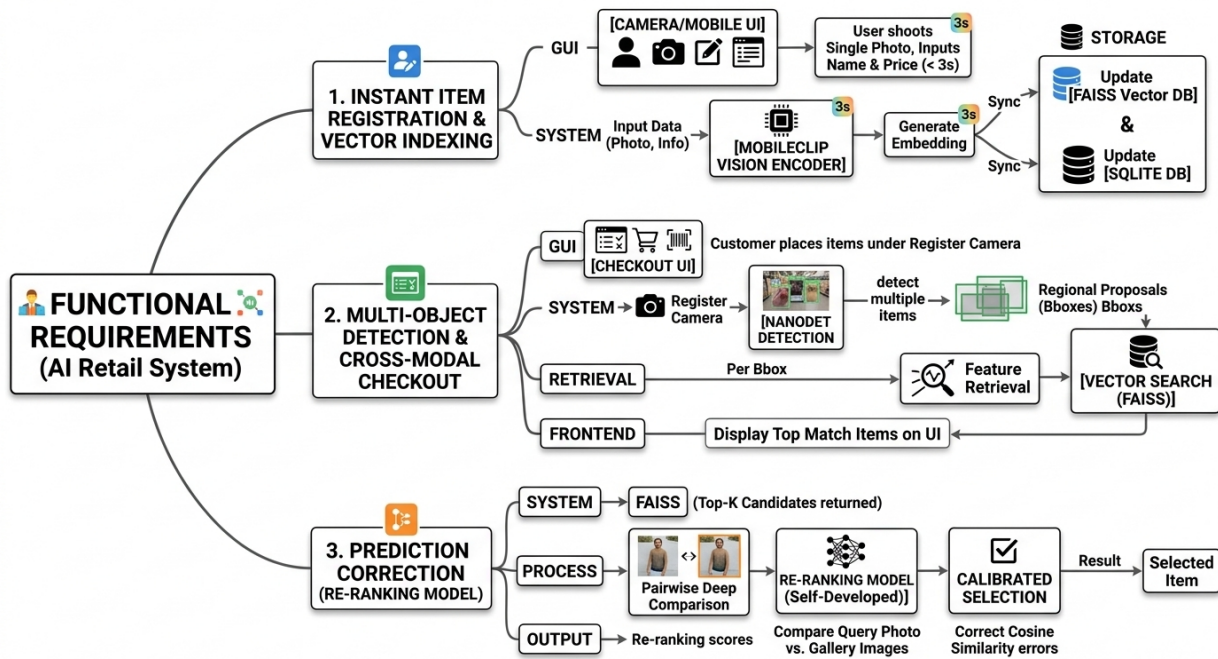


Fig. 2. Functional Requirements Diagram

- **FR1 : Instant Product Registration and Vector Index Archiving**

Users only need to take a single photo of a product and enter its name and price using their mobile devices. The system will generate an embedding within 3 seconds using an encoder and simultaneously update it to the vector database and the SQLite relational database.

- **FR2 : Multi-target Detection and Cross-modal Retrieval Checkout**

When a customer places an item under the camera at the checkout counter, the system automatically performs object detection, searches for features in each bounding box, and finally displays the item information with the highest matching degree on the front-end UI.

- **FR3: Prediction and Correction based on the Re-ranking Model**

After retrieving and returning the Top-K (, where $k = 5$) candidate products, the system must call the developer's self-developed Re-ranking (Matching) Model to perform a pairwise deep comparison between the Query image and the Gallery image in order to correct any misjudgments that may be caused by simple Cosine Similarity.

3.2 Non-Functional Requirements

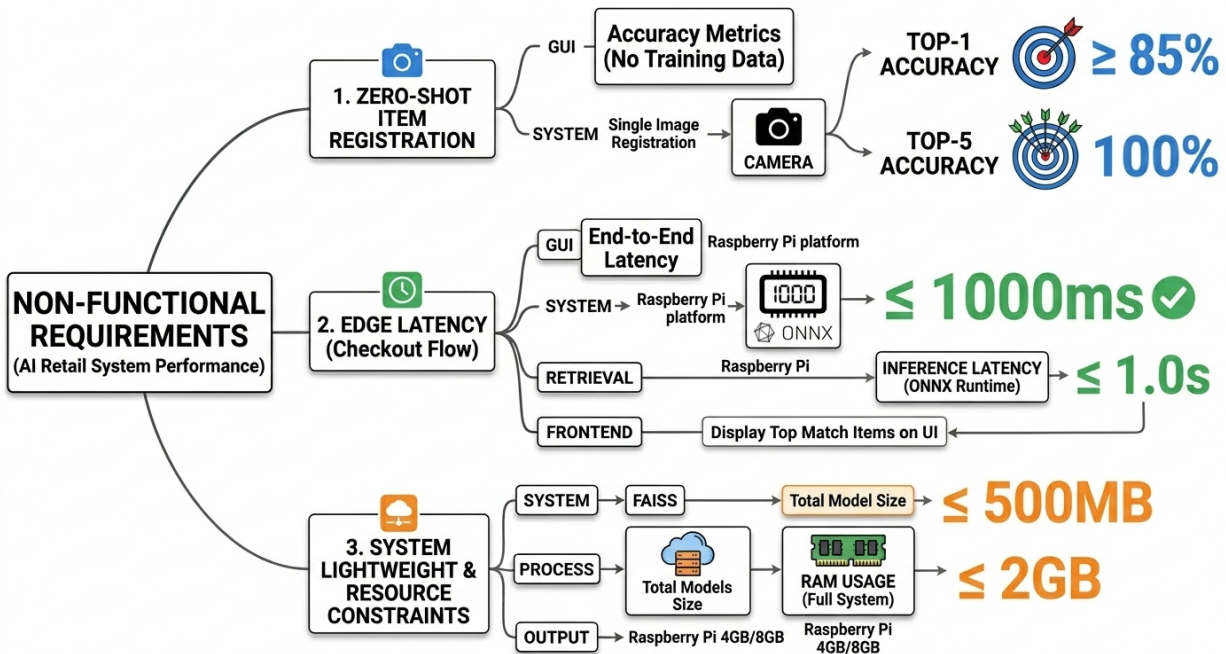


Fig. 3. Non-Functional Requirements Diagram

- **Instant Product Registration and Vector Index Filing**

The system must guarantee that when adding new, never-before-seen items, its recognition accuracy (zero-shot accuracy) is not significantly different from existing product accuracy, and the merchant's end does not need any labeled data or deep learning training knowledge.

Top 1 Accuracy $\geq 85\%$. Top 5 Accuracy = 100%

- **Low-Latency Interaction Experience at the Edge**

When the system is running on edge devices such as Raspberry Pi, the end-to-end latency from capturing images from the camera to displaying the checkout result must be less than 1000ms to avoid customers experiencing noticeable lag during checkout.

Inference Latency on Raspberry Pi (using ONNX Runtime) $\leq 1.0s$

- **Deployment Flexibility in Lightweight and Resource-Constrained Environments**

The memory usage of the entire system (including detection, encoding, and reordering models) must be kept within the hardware limits of the Raspberry Pi (e.g., 4GB/8GB RAM), and the models must be converted to ONNX format to maximize the utilization of the edge hardware accelerator.

Model Size $\leq 500MB$; RAM Usage $\leq 2GB$

4. System Design Overview

4.1 High-Level Architecture (UMLs)

USE CASE UML DIAGRAM

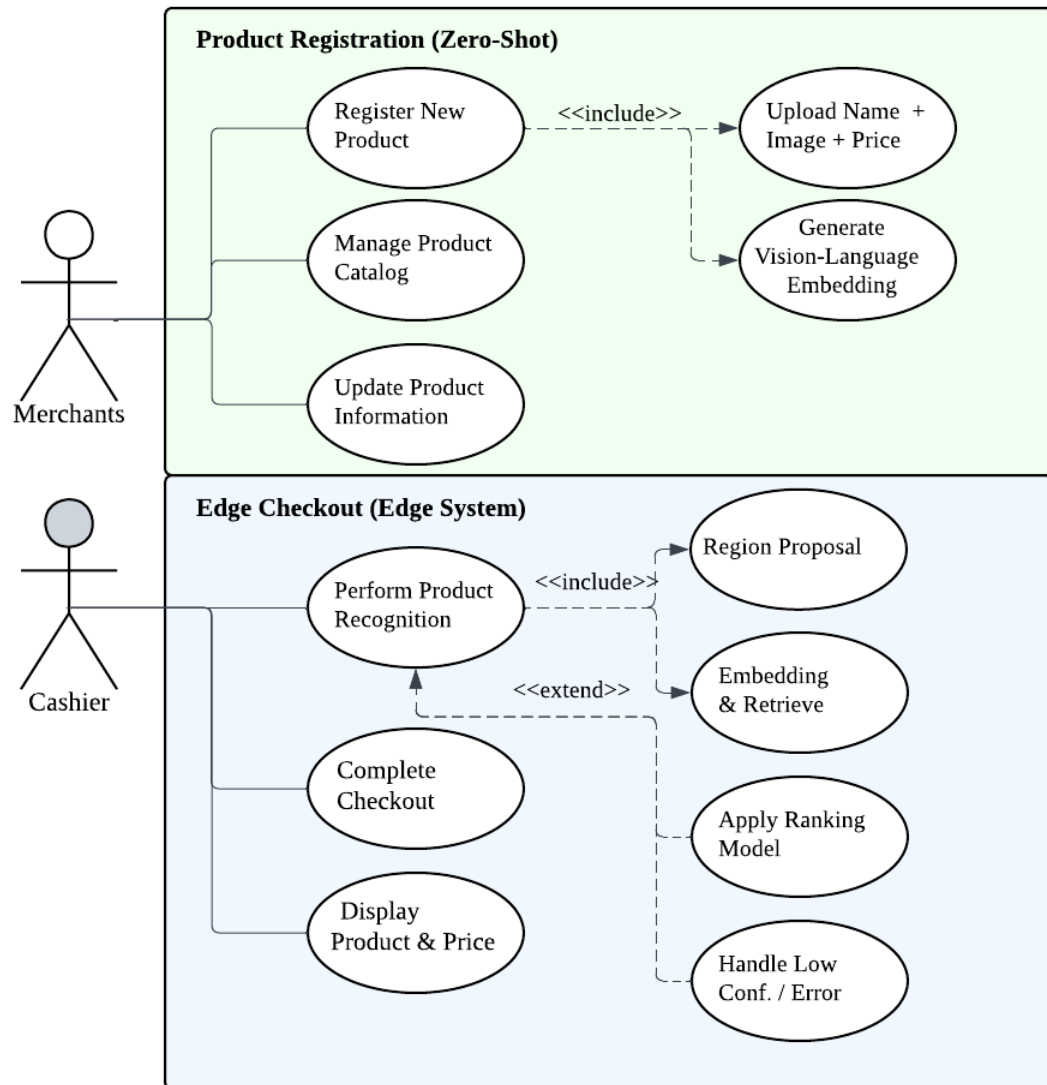


Fig. 4. Use Case Diagram of the Lightweight Vision-Language Zero-Shot Product Recognition System

This use case diagram illustrates the functional requirements of the proposed system, which is designed to help non-technical store owners rapidly build a zero-shot product recognition system for edge-device self-checkout. The diagram is divided into two main subsystems: **Product Registration** (upper section, green) and **Edge Checkout** (lower section, blue). In the **Product Registration** subsystem, the store owner (a non-technical merchant) can register a new product in a zero-shot manner by uploading a single product

image along with its name and price. This process involves generating vision-language embeddings using lightweight models such as **MobileCLIP** or **SigLIP**, and storing the embeddings together with their metadata in an SQLite-based vector database. Additional use cases, such as managing the product catalog and updating product information, support ongoing catalog maintenance.

In the **Edge Checkout** subsystem, the cashier interacts with a system deployed on a Edge Device to perform real-time product recognition. The workflow includes capturing an image from a camera, performing region proposal and cropping, generating embeddings, and retrieving the most similar product through cosine similarity matching. The system also integrates a reranking model trained by the author as an extension to improve both zero-shot and few-shot recognition accuracy when confidence is insufficient. If the confidence score falls below a predefined threshold, the "Handle Low Confidence/Error" use case allows for manual intervention. The identified product information is then displayed to complete the checkout process.

This design emphasizes zero-shot learning, eliminating the need to retrain the model whenever new products are added, while also supporting lightweight deployment suitable for resource-constrained edge devices. The separation between the registration and inference stages ensures ease of use for non-engineering users while maintaining real-time performance on edge hardware. The proposed reranking model represents the primary research contribution, aiming to improve retrieval accuracy without introducing additional latency.

SYSTEM ARCHITECTURE DESIGN

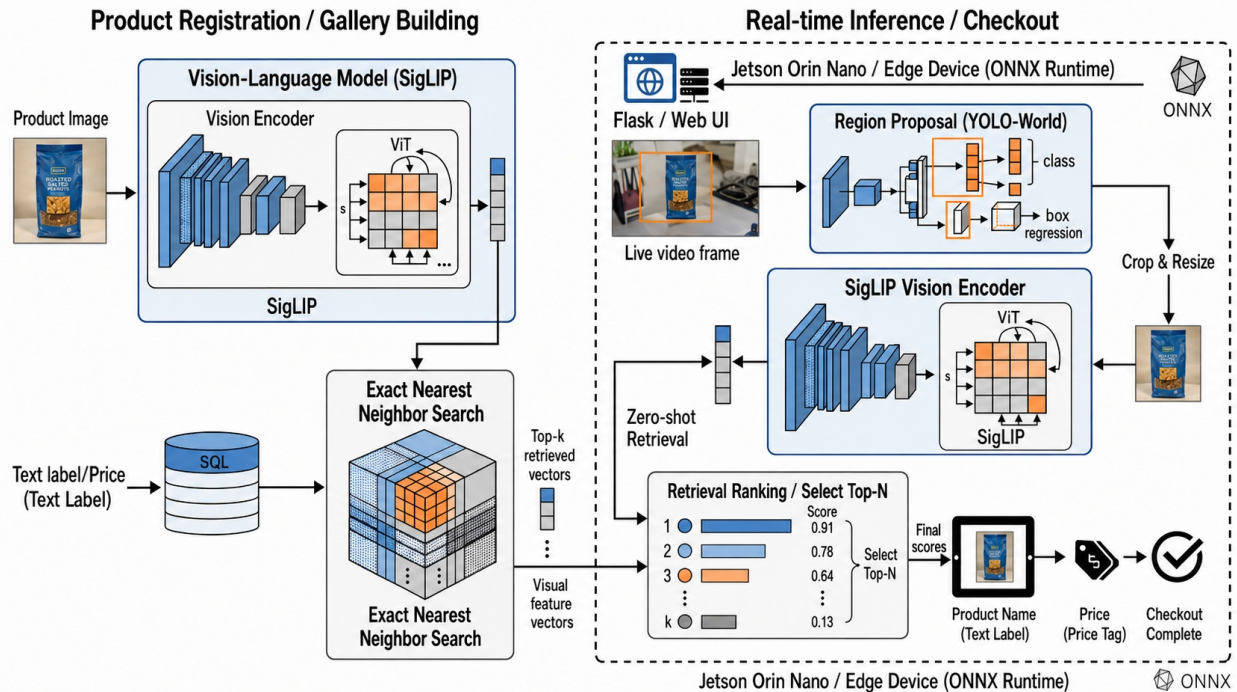


Fig. 5. System Architecture Design Diagram

This system adopts a modular, lightweight Vision-Language Retrieval architecture specifically designed for zero-shot product recognition on edge devices. It consists of two main stages: Product Registration (product registration/gallery construction) and Real-time Inference (real-time inference/checkout).

During the product registration stage, the merchant only needs to upload a single product image along with textual labels (product name and price). The system utilizes the **SigLIP** visual encoder to extract high-dimensional visual embedding vectors. These embeddings, together with the textual labels, are then stored in a hybrid database architecture: visual embeddings are stored in a Vector Database, while textual information and pricing data are stored in **SQLite**. The entire registration process requires no model fine-tuning, enabling a truly zero-shot setup.

During the real-time inference stage, the camera on the edge device continuously captures live images, which are first processed by **YOLO-World** to generate product bounding boxes (bboxes) through object detection. The detected product regions are then cropped and resized before being passed to **SigLIP** for visual embedding extraction. The resulting embeddings are compared against the registered product embeddings using **Exact Nearest Neighbor Search** with cosine similarity, enabling **zero-shot retrieval** and product identification. Once the most similar product is retrieved, the system accesses the corresponding product metadata and pricing information stored in SQLite to complete the self-checkout process. All models are converted to the **ONNX** format and deployed on the **Jetson Orin Nano**, ensuring low-latency inference and efficient execution suitable for resource-constrained edge environments.

ACTIVITY UML DIAGRAM

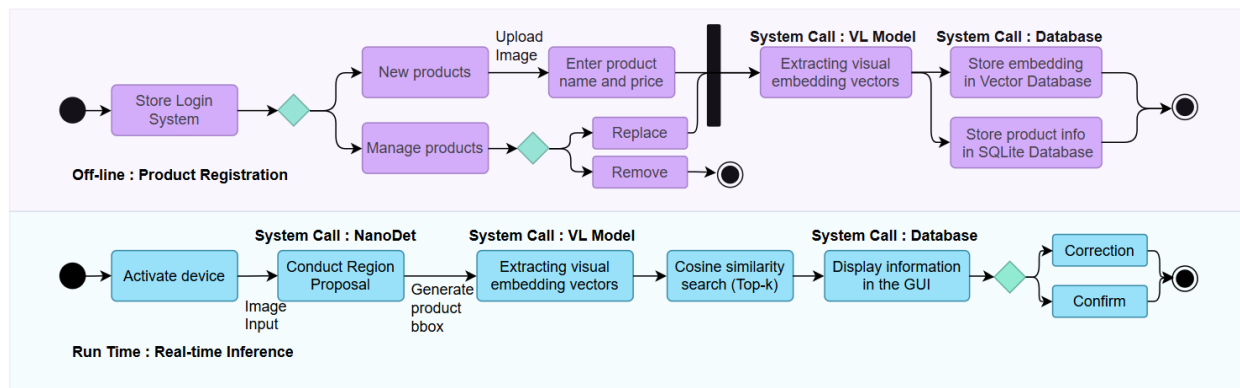


Fig. 6. User Activity Diagram

This diagram illustrates the two core user activity flows of the system. The upper part is the offline product registration process. After logging in, merchants can add, replace, or remove products. They only need to upload a single image and enter the name and price. The system automatically extracts the visual embedding through MobileCLIP and stores it in FAISS and SQLite, achieving zero-shot image library construction.

The second half describes the real-time inference process, executed on a Raspberry Pi 4 / Jetson Orin Nano. After generating product bounding boxes using NanoDet, the system extracts embedding vectors for cosine similarity retrieval, then uses a self-trained re-ranking model to improve accuracy, ultimately completing the self-checkout process. This activity diagram clearly demonstrates the system's ease of use and the real-time capabilities of Edge AI.

4.2 ML System Design & MLOps

Object Detection Model

The proposed system adopts **YOLO-World with a YOLOv8-s architecture** for edge-based product detection. Its open-vocabulary capability enables **zero-shot object detection**, allowing users to define or modify target product categories through text prompts without retraining the model. The system focuses on detecting product locations and bounding-box dimensions while maintaining a balance between inference speed and accuracy for deployment on devices such as Jetson Orin Nano, Raspberry Pi and RK3568.

The model will be exported to ONNX for cross-platform deployment and further optimized through quantization and hardware-specific acceleration. Future development will introduce an MLOps workflow using Git, DVC, GitHub Actions, and MLflow for data versioning, automated testing, model conversion, deployment, and experiment tracking. Additional developer-oriented optimizations will include configurable prompts, model replacement interfaces, performance monitoring, and deployment profiling to improve maintainability, reproducibility, and scalability.

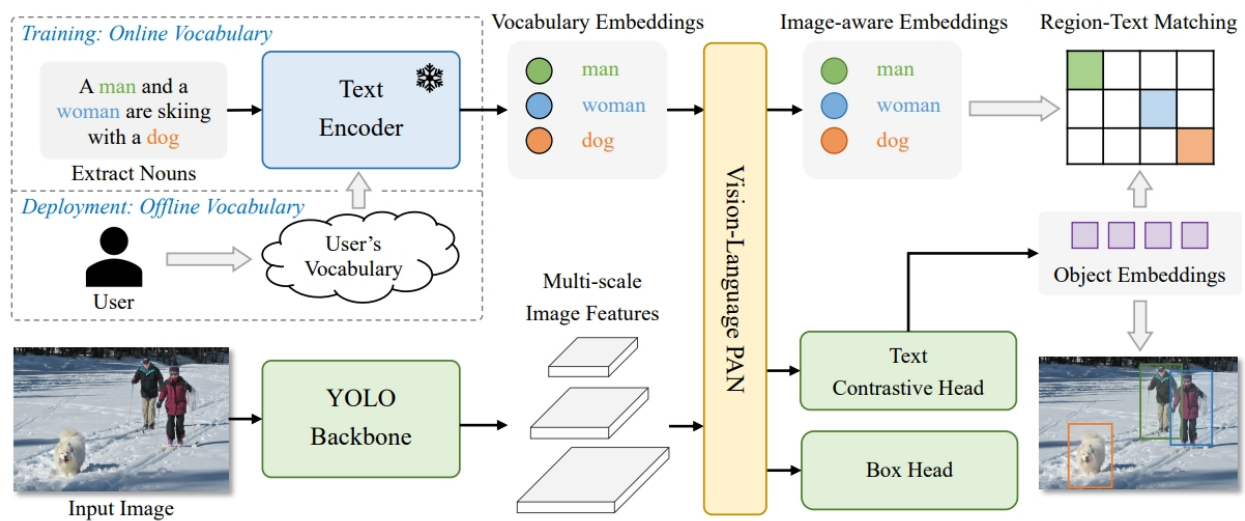


Fig 7. YOLO-World Zero-shot Object Detection Model Architecture (from “YOLO-World: Real-Time Open-Vocabulary Object Detection”[])

YOLO-World achieves efficient open-vocabulary detection by combining the real-time YOLO architecture with vision-language pre-training. Its RepVL-PAN integrates text embeddings into multi-scale visual features through text-guided feature fusion, enabling the detector to associate image regions with user-defined semantic concepts. During training, region-text contrastive learning increases the similarity between matched object regions and text descriptions while separating unrelated pairs. The model is pre-trained using large-scale object-detection, visual-grounding, and image-text datasets, which improves semantic generalization to unseen categories. In addition, its prompt-then-detect strategy allows text embeddings to be encoded and re-parameterized before inference, avoiding repeated language-model computation and preserving real-time detection efficiency (Cheng et al., 2024).

Vision-Language Embedding Model

OpenAI introduced CLIP (Contrastive Language-Image Pre-training) in 2021, using 400 million image-text pairs and contrastive learning (contrastive loss) to align images and text into a shared embedding space, enabling strong zero-shot capabilities and laying the foundation for Vision-Language Models (VLMs). From 2021 to 2022, models such as ALIGN and Florence expanded dataset scale and optimized architectures. In 2023, Google's SigLIP replaced softmax with sigmoid loss, addressing batch memory bottlenecks and improving both training efficiency and performance. During 2024–2025, MobileCLIP (Apple) was optimized for edge deployment, combining a lightweight backbone with multi-modal reinforced training to achieve 3–15 ms latency and high zero-shot accuracy; SigLIP 2 further enhanced multilingual support and dense feature learning. From 2026 to the present, lightweight VLMs have continued evolving toward lower parameter counts and higher efficiency.

This project adopts MobileCLIP / SigLIP as the core embedding models to enable zero-shot product recognition without retraining. We compare CLIP, SigLIP, and MobileCLIP in terms of accuracy, latency, and few-shot performance, and train a re-ranking model to improve recognition accuracy. The design combines lightweight deployment with recent vision-language (VL) technologies for practical edge AI retail applications.

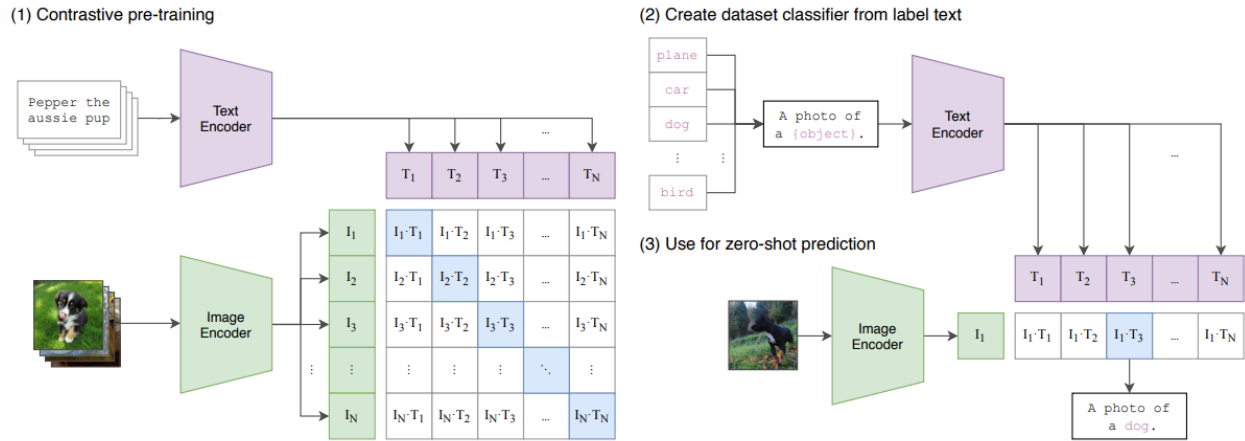


Fig. 8. Standard image models jointly train an image feature extractor and a linear classifier to predict labels, whereas CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training samples. (Adapted from "Learning Transferable Visual Models From Natural Language Supervision")

Feasibility Testing

Table. 1. Grocery Store Packages Dataset Comparison (31 Classes , 833 test images)

Embedding Model	Top-1 Accuracy	Top-5 Accuracy	Avg. Similarity
SigLIP	72.27%	94.96%	84.38%
MobileCLIP	59.90%	88.00%	73.38%
CLIP	51.86%	81.63%	81.94%
ResNet	39.74%	67.95%	81.52%

4.3 Technology Stack

Layer	Technology	Illustrate
Frontend	HTML/CSS/JS	Simple and intuitive Web UI
Backend	Flask	High-performance asynchronous framework
Embedding Model	SigLIP (ONNX)	Vision-Language Model
Region Proposal	YOLO-World (ONNX)	Object Detection Model
Database	SQLite	Lightweight, zero settings
Vector Database	Exact Nearest Neighbor Search	High-performance vector search library
Development & Deployment Tools	Git, Docker, Docker Compose, PyTorch, ONNX Converter	Version control, containerized deployment, model conversion, and experiment reproducibility.
Operating System	Windows	Universality
Support Equipment	Jetson Orin Nano / Raspberry Pi	Commonly used and excellent edge computing devices

4.4 GUI Design

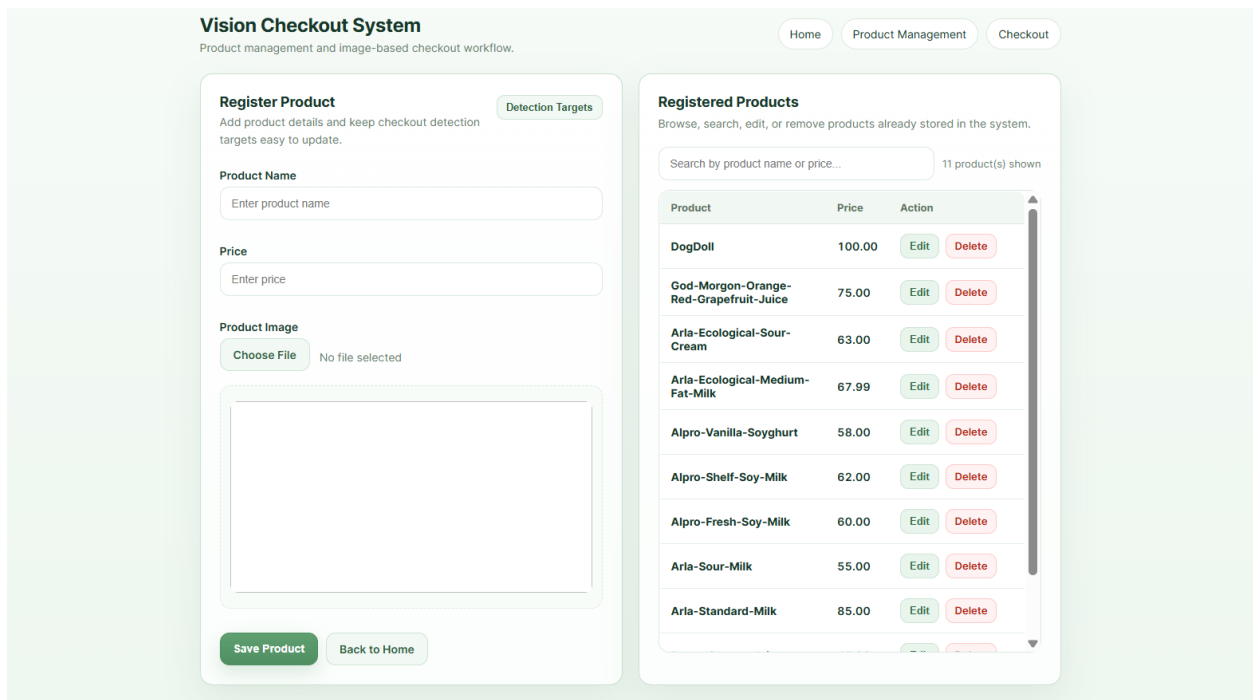


Fig. 9. Storefront GUI Design Showcase

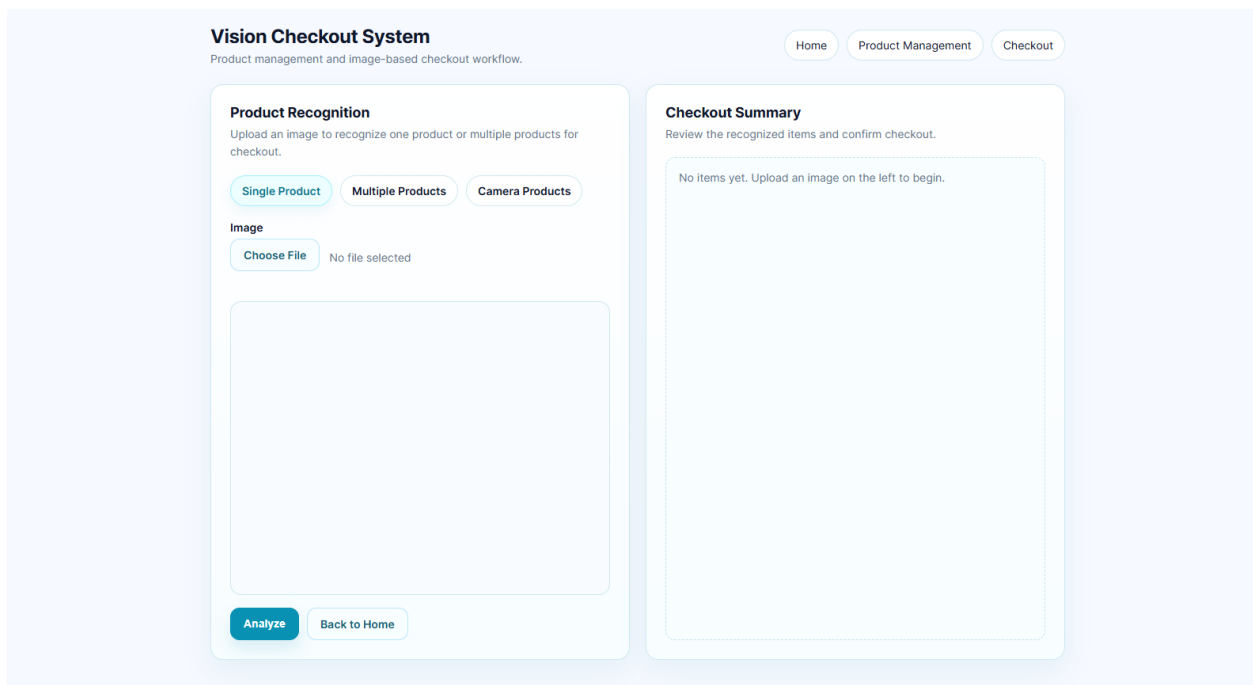


Fig. 10. Checkout GUI Design Showcase

5. Development Estimation and Release Plan

The development plan is divided into three stages: the completed Minimum Viable Product, a short-term Version 2 improvement release, and a long-term Version 3 mature release.

The estimation considers calendar time, available manpower, engineering effort, equipment cost, and technical uncertainty. Unlike a full-time commercial project, this system is developed by undergraduate students alongside academic and personal commitments. After the completion of the MVP, the project is expected to gradually transition from a three-person team to one primary maintainer. The primary developer is assumed to contribute approximately six hours per week through GitHub, while the other two members may provide occasional support or leave the project due to practical constraints.

One person-month is defined as approximately 160 working hours. Therefore, the estimated calendar duration may be significantly longer than the equivalent engineering effort. The estimation also follows the principle of *The Mythical Man-Month*: adding contributors does not reduce the schedule proportionally because coordination, code review, testing, and integration require additional effort.

The estimates include not only feature development, but also refactoring, testing, documentation, model evaluation, deployment, and maintenance. However, the expected result remains an academic and open-source software release rather than a fully certified commercial product.

Stage	Duration	Main Objective
Version 1.0 — MVP	Completed	Validate the feasibility of zero-shot product recognition on edge devices
Version 2.0 — Professional Release	3 months	Improve reliability, accuracy, maintainability, and deployment consistency
Version 3.0 — Mature Release	3– ∞ months	Provide one-click installation and a complete user experience

5.1 Version 1.0 — Minimum Viable Product

Version 1.0 has been completed as a technical feasibility prototype. It provides product registration, image embedding generation, similarity retrieval, region proposal, product recognition, price display, and a basic checkout workflow.

The purpose of this stage was to confirm that lightweight Vision-Language Models could support zero-shot product recognition on edge devices. The MVP demonstrates the complete recognition pipeline, but it still depends on a development-oriented installation process and has limited automated testing, hardware compatibility, failure recovery, and real-world validation.

Because development records were not tracked using a formal time-reporting system, the effort of Version 1.0 should be treated as a retrospective estimate rather than an exact measurement. Its main value is the completed proof of concept and the technical foundation for future releases.

5.2 Version 2.0 — Professional Release

Version 2.0 is estimated to require approximately three calendar months. One developer will act as the primary maintainer and contribute about six hours per week, while the other two members may provide occasional support when available.

The estimated total effort is approximately 80 to 100 working hours. The scope is therefore limited to improving the existing MVP rather than redesigning the entire system. The main work includes basic architectural refactoring, improved product registration, more stable recognition workflows, clearer error handling, limited automated testing, and simplified deployment on a development computer or Jetson Orin Nano.

Recognition quality may be improved through better preprocessing, multiple reference images, confidence thresholds, and basic reranking. Large-scale model training, extensive dataset collection, and broad hardware compatibility testing are outside the expected scope.

The only expected direct cost is the Jetson Orin Nano development device. No separate labor cost is included because development will be conducted as an individual academic and open-source project.

Version 2.0 will be considered complete when the system is more stable than the MVP, installation steps are documented, common errors can be handled, and the system can be demonstrated repeatedly on at least one computer environment and one Jetson Orin Nano configuration.

5.3 Version 3.0 — Mature Release

The duration of Version 3.0 is currently undetermined. Development is expected to continue mainly through one primary developer contributing approximately six hours per week, equivalent to about 300 working hours per year.

The objective is to provide a more complete user experience in which users can download the correct package, install the software without manually configuring Python or Git, connect a supported camera, register products, and begin using the system through a guided interface.

The estimated engineering effort is approximately 400 to 700 hours. At the expected development rate, this corresponds to roughly 16 to 27 months of active development. The actual calendar duration may be longer because development may be interrupted by graduation, employment, hardware availability, or dependency changes.

To keep the scope realistic, Version 3.0 should initially support only a limited number of officially tested environments, such as one desktop platform and one Jetson Orin Nano configuration. The main work includes installation packaging, dependency management, hardware detection, setup guidance, diagnostics, backup and recovery, documentation, and compatibility testing.

No labor cost is estimated for this stage. The main direct cost remains the Jetson Orin Nano and any necessary accessories used for deployment and testing.

Version 3.0 will be considered successful when a non-technical user can install and operate the system on an officially supported platform with minimal assistance. The expected result is a practical and user-friendly open-source release rather than a fully certified commercial product.

6. Testing

6.1 Testing Objectives and Strategy

The testing process aims to determine whether the proposed system satisfies its defined functional and non-functional requirements while remaining technically feasible for deployment on resource-constrained edge devices. The evaluation covers the complete operational pipeline, including product registration, image validation, embedding generation, object-region proposal, similarity-based retrieval, product information lookup, checkout calculation, user correction, and exception handling. In addition to verifying individual functions, the testing process examines whether data and control flow remain consistent across the frontend, backend, AI models, vector index, and SQLite database.

Because Version 1.0 is an academic Minimum Viable Product, the current testing strategy emphasizes functional testing, integration testing, system-level validation, and qualitative feasibility analysis rather than full commercial certification. Model performance is evaluated using Top-1 and Top-5 retrieval accuracy, supported by qualitative inspection of object-detection and recognition results. Deployment validation compares system behavior across a GPU-enabled development laptop and resource-constrained edge platforms to identify compatibility, performance, and resource-usage limitations.

The test results are used not only to determine whether a function passes or fails, but also to identify risks that may affect practical operation, such as ranking ambiguity, detection failure, inconsistent database records, invalid inputs, and hardware-dependent latency. Automated unit, regression, and continuous-integration testing remain limited in Version 1.0 and are planned as part of the Version 2.0 reliability and maintainability improvements.

6.2 Test Environment

The system was evaluated on three hardware environments representing development, accelerated edge deployment, and low-resource deployment.

Platform	Main Specification	Testing Purpose
Development Laptop	x86-based laptop, NVIDIA GeForce RTX 4060 Laptop GPU, Windows environment	Model evaluation, system development, integration testing, and baseline inference
Jetson Orin Nano	ARM-based NVIDIA edge device with integrated GPU acceleration and Linux/JetPack environment	ONNX deployment, hardware-accelerated inference, and edge-system validation
Raspberry Pi 4	Quad-core ARM CPU, limited memory, Linux environment, without dedicated GPU	Compatibility testing and evaluation under constrained computing resources

The software environment includes Python, Flask, SQLite, PyTorch or ONNX Runtime, a Vision-Language embedding model, and a zero-shot object detection model. Test inputs include the Grocery Store Dataset and locally captured product images.

6.3 Functional Testing

Test ID	Requirement	Test Scenario	Expected Result	Actual Result	Status
FT-01	FR1	Register a product with valid name, price, and image	Product metadata and embedding are stored successfully	Successfully stored	Pass
FT-02	FR1	Register a product without an image	System rejects the request	Error message displayed	Pass
FT-03	FR1	Enter an invalid price	System rejects invalid input	Input validation triggered	Pass
FT-04	FR2	Upload an image containing a registered product	Correct product and price are displayed	Retrieved correctly	Pass
FT-05	FR2	Add a recognized product to checkout	Item appears in checkout list and total is updated	Correct total shown	Pass
FT-06	FR2	Remove an item from checkout	Item is removed and total is recalculated	Correctly updated	Pass
FT-07	FR3	Retrieval confidence is low	Top-k candidates or correction UI is shown	Candidate list displayed	Pass
FT-08	Database	Delete a registered product	Metadata and vector entry are removed	Removed successfully	Pass

6.4 Functional Testing in Show Case

Register Product

Add product details and keep checkout detection targets easy to update.

Product Name
GHIRARDELLI Double Chocolate Hot Cocoa Mix

Price
7.99

Product Image
Choose File 81WRP+KWwqL...SX679_.jpg



Save Product **Back to Home**

Registered Products

Browse, search, edit, or remove products already stored in the system.

Search by product name or price: 11 product(s) shown

Product	Price	Action
DogDoll	100.00	Edit Delete
God-Morgon-Orange-Red-Grapefruit-Juice	75.00	Edit Delete
Aria-Ecological-Sour-Cream	63.00	Edit Delete
Aria-Ecological-Medium-Fat-Milk	67.99	Edit Delete
Alpro-Vanilla-Soyghurt	58.00	Edit Delete
Alpro-Sheif-Soy-Milk	62.00	Edit Delete
Alpro-Fresh-Soy-Milk	60.00	Edit Delete
Aria-Sour-Milk	55.00	Edit Delete
Aria-Standard-Milk	85.00	Edit Delete

Product Recognition

Upload an image to recognize one product or multiple products for checkout.

[Single Product](#) [Multiple Products](#) [Camera Products](#)

Image
Choose File No file selected



Click a numbered bounding box on the image to correct its classification.

Analyze **Back to Home**

Checkout Summary

Review the recognized items and confirm checkout.

GHIRARDELLI Double Chocolate Hot Cocoa Mix x1
Price: 7.99
Subtotal: 7.99

Total 7.99

Complete Checkout

This section illustrates a simulated usage scenario. When a user inputs information about a registered product, including its name, price, and image, it can be immediately imported into the system's database. Registered products are then displayed in the list on the right side of the registration page. The product recognition function demonstrates a practical example of object detection and product categorization to test its feasibility. The detection results and checkout results are then displayed in the list on the right.

6.5 Model and Retrieval Evaluation

6.5.1 Embedding-based retrieval zero-shot product recognition

Grocery Store Packages Dataset Comparison (31 Classes , 833 test images)

Embedding Model	Top-1 Accuracy	Top-5 Accuracy	Avg. Similarity
SigLIP	72.27%	94.96%	84.38%
MobileCLIP	59.90%	88.00%	73.38%
CLIP	51.86%	81.63%	81.94%
ResNet	39.74%	67.95%	81.52%

SigLIP achieved the strongest retrieval performance among the evaluated models, with a Top-1 accuracy of 72.27% and a Top-5 accuracy of 94.96%. The high Top-5 result indicates that the learned embedding space can reliably place the correct product among a small candidate set, supporting the technical feasibility of embedding-based zero-shot retrieval. However, the Top-1 performance remains below the specified acceptance target, showing that visually similar packaging and limited reference images still cause ranking ambiguity. From an engineering perspective, the current model is suitable for an assisted-recognition workflow with confidence filtering, Top-K candidate presentation, or re-ranking, but it is not yet sufficiently reliable for fully automatic checkout without user confirmation.

6.5.2 Zero-shot object detection product proposal system



The zero-shot object detection module successfully generated product-region proposals across different product types and viewing conditions without category-specific retraining. The qualitative results demonstrate that the detector can provide usable bounding boxes for downstream cropping and embedding retrieval, reducing background interference and supporting multi-object processing. However, the current demonstration should be considered a feasibility validation rather than a complete detection benchmark, since localization accuracy, missed detections, false positives, and edge-device latency have not yet been evaluated quantitatively. From a system perspective, the module is suitable as a region-proposal component, while confidence thresholds and fallback handling remain necessary to prevent detection errors from propagating through the retrieval pipeline.

6.6 Integration Testing

6.6.1 CRUD Flask front-end and back-end framework

After testing the actual deployment of the CRUD Flask architecture on the local machine, it was confirmed that the message "GET / HTTP/1.1" 200 .

6.6.2 Product registration system integration

Register Product

Add product details and keep checkout detection targets easy to update.

Product Name

Unite Milky Tea 300ml

Price

10

Product Image

Choose File

milkytea.jpg



Save Product

Back to Home

Detection Targets

Set what the checkout camera should look for. Use everyday words such as bottle, box, bread, cup, or snack package.

Add a target, e.g. bread or lunch box

Add

product

Camera's target

Delete

retail product

Camera's target

Delete

packaged product

Camera's target

Delete

package

Camera's target

Delete

These targets are saved in the database and applied to multi-product checkout detection.

Done

Registered Products

Browse, search, edit, or remove products already stored in the system.

Search by product name or price.

12 product(s) shown

Product	Price	Action
GHIRARDELLI Double Chocolate Hot Cocoa Mix	7.99	Edit Delete
DogDoll	100.00	Edit Delete
God-Morgan-Orange-Red-Grapefruit-Juice	75.00	Edit Delete
Arla-Ecological-Sour-Cream	63.00	Edit Delete
Arla-Ecological-Medium-Fat-Milk	67.99	Edit Delete
Alpro-Vanilla-Soyghurt	58.00	Edit Delete
Alpro-Shelf-Soy-Milk	62.00	Edit Delete
Alpro-Fresh-Soy-Milk	60.00	Edit Delete
Arla-Sour-Milk	55.00	Edit Delete

Edit Product

Update the product name or price. Changes take effect immediately after confirmation.

Product Name

Alpro-Vanilla-Soyghurt

Price

58.0

Tip: press Esc to close this window.

Cancel

Confirm Changes

This section showcases the user interface and functional design of the entire Product registration system integration page. After registering product information, prices, and corresponding display images, users can press "Save" to directly convert embeddings and save product information to the database. For actual camera and image usage, the provided Region Proposal object detection system offers a customizable interface where users can input item names to adjust the detection system designed for different environments and item types. Users can manage updates and deletions in their system using the registered product database interface shown in the image; they can edit names, product prices, or remove items.

6.6.3 Product recognition system integration

Vision Checkout System

Product management and image-based checkout workflow.

Home

Product Management

Checkout

Product Recognition

Use the camera to capture a checkout image. The frame is sent to the same multi-product AI pipeline only after you press Capture & Analyze.

Single Product

Multiple Products

Camera Products

The live preview stays on the left. Press the button below to freeze one frame and send only that frame to checkout recognition.



Capture & Analyze

Back to Home

Captured Result

AI output for the last captured frame appears here.



Click a numbered bounding box on the result image to correct its classification.

Checkout Summary

Review the recognized items and confirm checkout.

DogDoll

Price: 100.00

Subtotal: 200.00

Total

200.00

Complete Checkout

Vision Checkout System

Product management and image-based checkout workflow.

Home

Product Management

Checkout

Product Recognition

Upload an image to recognize one product or multiple products for checkout.

Single Product

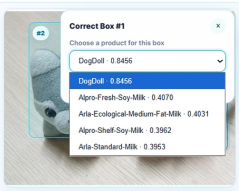
Multiple Products

Camera Products

Image

Choose File

No file selected



Click a numbered bounding box on the image to correct its classification.

Analyze

Back to Home

Checkout Summary

Review the recognized items and confirm checkout.

DogDoll

Price: 100.00

Subtotal: 200.00

Total

200.00

Complete Checkout

This section showcases the user interface and functional design provided on the entire Product Recognition System Integration page. Users can upload images or connect using a camera device. The region proposal and classification results of the recognition are presented to the user. If the classification is incorrect, the user can correct it from the top 5 candidate solutions. Finally, the entire automated checkout calculation results will be displayed on the right side.

6.7 Robustness and Error-Handling Testing

Scenario	Expected System Behavior
Blurred image	Show low-confidence result or request recapture
Product partially occluded	Return candidate list or reject uncertain result
Unknown product	Do not force an incorrect high-confidence match
Similar packaging	Return Top-k candidates for manual correction
Unsupported image format	Reject input with a clear message
Camera disconnected	Display device error without crashing
Database unavailable	Stop registration and preserve consistency
Model file missing	Display initialization error
Empty product database	Prompt the merchant to register products
Duplicate product registration	Warn or allow controlled update
Low-quality camera equipment	Affected by light and resolution interference.

Robustness testing evaluates whether the system can handle invalid inputs, uncertain predictions, hardware failures, and incomplete system states without crashing or producing misleading checkout results. Tested or considered scenarios include blurred images, partial occlusion, visually similar packaging, unknown products, unsupported image formats, camera disconnection, missing model files, unavailable databases, and empty product catalogs.

When prediction confidence is insufficient, the system should avoid automatically accepting the Top-1 result and instead return Top-K candidates or request manual confirmation. Input validation and exception handling are used to preserve consistency between product metadata and vector records. The current MVP provides basic handling for common failures, while more comprehensive recovery mechanisms and diagnostic reporting remain future work.

6.8 Regression and Automated Testing

Version 1.0 mainly relies on manual functional and integration testing because the system is still an MVP. Core workflows are repeatedly verified after major changes, including product registration, database operations, embedding generation, retrieval, checkout calculation, and user correction.

Future releases will introduce automated tests for Flask APIs, SQLite CRUD operations, vector-index consistency, embedding dimensions, price calculation, and invalid-input handling. A fixed validation dataset will also be used to detect model-performance regression. These tests may be integrated with GitHub Actions so that code changes can be checked automatically before release. Accuracy degradation, unexpected API failures, or significant latency increases will be treated as regression conditions requiring review.

7. Version History

Version	Date	Description	Dev. Log
0.0.1	4/22/2026	Initial draft	After completing the VLM feasibility test, basic front-end and back-end architecture design and internal coordination, Also took a trip to the edge of the known universe, during which I accidentally drank too much coffee.
0.0.2	5/15/2026	Initial integration	Preliminary architecture integration and module performance testing. This version added a lot of code, and I watched many cute cat and fox videos in the process.
1.0.0	6/24/2026	Minimum Viable Product	The MVP for the project has been completed and officially released on GitHub. Go through the super busy end-of-term period. The test engineers on the bar project were very satisfied with the results, but a customer walked into the bar and ordered a plate of fried rice, and the bar burst into flames.

Test Dataset : <https://github.com/marcusklason/GroceryStoreDataset>